UNIT-3

Introduction to Number Theory

Prime Numbers:

A prime number is a whole number greater than 1 that has exactly two factors:

- **1**
- **The number itself**

Examples: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47...

1. Fermat's Little Theorem (Number Theory)

If $p$is a prime number and $a$is not divisible by p, then

$$a^{p-1} \equiv 1(\mathrm{mod}p)$$

$$a^{p-1} \equiv 1(\mathrm{mod}p)$$

Example: For $a = 2$and $p = 7$,

$$2^6 = 64 \equiv 1(\mathrm{mod}7).$$

Example: $a = 3, p = 5$

Since **5 is prime** and 5 does not divide 3:

$$3^{5-1} \equiv 1 \pmod 5$$

$$3^4 = 81$$

Now divide 81 by 5:

$$81 = 5(16) + 1$$

The remainder is **1.**

Fermat's Little Theorem: Step-by-Step example

Given:

$$a = 2,\ p = 7$$

Since **7 is a prime number**, Fermat's Little Theorem applies.

Step 1: Write the formula

Fermat's Little Theorem says:

$$a^{p-1} \equiv 1 \pmod{p}$$

Step 2: Substitute $a = 2$ and $p = 7$

$$2^{7-1} \equiv 1 \pmod{7}$$

Step 3: Calculate the exponent

$$7 - 1 = 6$$

So:

$$2^6 \equiv 1 \pmod{7}$$

Step 4: Calculate 2^6

$$2^6 = 2 \times 2 \times 2 \times 2 \times 2 \times 2$$

$$= 64$$

Therefore:

$$64 \equiv 1 \pmod 7$$

Step 5: Check the remainder

Divide 64 by 7:

$$64 \div 7 = 9 \text{ remainder } 1$$

because

$$7 \times 9 = 63$$

and

$$64 - 63 = 1$$

Therefore:

$$\boxed{2^6 \equiv 1 \pmod 7}$$

In simple words: When 64 is divided by 7, the remainder is **1**, so the statement is true.

This theorem is widely used in cryptography and modular arithmetic.

Euler's Theorem (Number Theory)

Euler's theorem is a generalization of Fermat's Little Theorem. If $a$and $n$are positive integers such that gcd(a, n) = 1, then

$$a^{(n)} \equiv 1(\mathrm{mod} n)$$

where $\varphi(n)$is the Euler totient function, which counts the positive integers less than $n$that are relatively prime to n.

$$a^{(n)} \equiv 1(\mathrm{mod} n)$$

Example

Take a = 3and n = 10.

- Numbers less than 10 and coprime to 10 are: 1, 3, 7, 9
- Therefore, $\varphi(10)$ = 4

Applying Euler's theorem:

$$3^4 \equiv 1(\mathrm{mod} 10)$$

Since 3^4 = 81,

$$81 \equiv 1(\mathrm{mod} 10)$$

which is true.

Relation to Fermat's Little Theorem

If n = $p$is a prime number, then

$$(p) = p - 1$$

and Euler's theorem becomes

$$a^{p-1} \equiv 1 (\mathrm{mod} p),$$

which is exactly **Fermat's Little Theorem**.

Applications

- Modular arithmetic calculations
- Cryptography, especially the RSA Algorithm
- Finding modular inverses
- Solving congruence

Testing for Primality

A **primality test** is a method used to determine whether a positive integer $n > 1$ is **prime** or **composite**.

Basic Method: Trial Division

To test whether n is prime:

1. Check if n is divisible by 2.
2. If not, test divisibility by odd numbers up to $\sqrt{n}$.
3. If no divisor is found, n is prime.

Example: Test $n = 29$

$$\sqrt{29} \approx 5.38$$

Check divisibility by 2, 3, and 5:

- $29 \not\equiv 0 (\mathrm{mod} 2)$
- $29 \not\equiv 0 (\mathrm{mod} 3)$

- $29 \equiv /0 (\text{mod} 5)$

Therefore, 29 is prime.

Fermat Primality Test

Based on **Fermat's Little**

Theorem: If $p$is prime and gcd(a,

p) = 1, then

$$a^{p-1} \equiv 1 (\text{mod} p)$$

$$a^{p-1} \equiv 1 (\text{mod} p)$$

To test a number n:

1. Choose an integer $a$with $1 < a < n$.
2. Compute $a^{n-1} \, mod \, n$.
3. If the result is not 1, $n$is definitely composite.
4. If the result is 1, $n$is **probably**

prime. Example: $n = 15, \; a = 2$

$$2^{14} \, mod \, 15 = 4 \neq 1$$

Therefore, 15 is composite.

Euler Primality Test

Using Euler's theorem, for an odd prime p,

$$^{(p-1)/2} \equiv \pm 1 (\text{mod} p)$$

A failure of this condition proves the number is composite.

Miller–Rabin Primality Test

The most widely used practical probabilistic

test. For an odd integer n:

1. Write $n - 1 = 2^s d$, where $d$is odd.
2. Choose a base a.
3. Check certain modular exponentiation

conditions. If the test fails, $n$is composite.
If it passes several rounds with different bases, $n$is prime with extremely high probability.

This test is commonly used in cryptographic systems because it is much faster than trial division for large numbers.

Comparison

Method	Speed	Accuracy
Trial Division	Slow for large n	Exact
Fermat Test	Fast	Can be fooled by some composites
Euler Test	Better than Fermat	Still probabilistic
Miller–Rabin	Very fast	Extremely reliable

Key Idea

- **One counterexample** is enough to prove a number is composite.
- Repeated successful tests increase confidence that a number is prime.
- Modern cryptographic systems typically use the **Miller–Rabin primality test** to generate large prime numbers.

Get smarter responses, upload files and images, and more.

Chinese Remainder Theorem (CRT)

The **Chinese Remainder Theorem** is a result in number theory that allows a system of congruences to be solved simultaneously when the moduli are pairwise coprime.

Statement

Suppose

$$x \equiv a_1 (\text{mod } m_1)$$
$$x \equiv a_2 (\text{mod } m_2)$$

where $m_1, m_2, \dots, m_k$ are pairwise coprime:

$$\gcd(m_i, m_j) = 1 (i \neq j)$$

Then there exists a **unique solution modulo**

$$M = m_1 m_2 \cdots m_k.$$

Example

Solve:

$$x \equiv 2 (\text{mod} 3)$$
$$x \equiv 3 (\text{mod} 5)$$
$$x \equiv 2 (\text{mod} 7)$$

Since 3, 5, and 7 are pairwise coprime, CRT applies. The solution is:

$$x \equiv 23 (\text{mod } 105)$$

because:

- $23 \equiv 2 (\text{mod} 3)$
- $23 \equiv 3 (\text{mod} 5)$
- $23 \equiv 2 (\text{mod} 7)$

Thus all solutions are $x = 23 + 105k, k \in \mathbb{Z}$.

Applications

1. **Fast modular arithmetic**
2. **Cryptography**, especially the RSA Algorithm
3. Solving systems of linear congruences
4. Computer algebra systems
5. Distributed computations and coding theory

Discrete Logarithms

The **discrete logarithm problem (DLP)** is the modular-arithmetic analogue of ordinary logarithms.

Definition

Suppose

$$a^x \equiv (\mathrm{mod} n).$$

The **discrete logarithm** of $b$to the base $a$modulo $n$is the exponent $x$satisfying the congruence.
We write

$$x = \log_a (\mathrm{mod} n).$$

Example

Find $x$such that

$$2^x \equiv 8(\mathrm{mod}11).$$

Compute powers of 2 modulo 11:

$$2^1 \equiv 2(\mathrm{mod}11)$$
$$2^2 \equiv 4(\mathrm{mod}11)$$
$$2^3 \equiv 8(\mathrm{mod}11)$$

Therefore,

$$x = 3.$$

So the discrete logarithm of 8 base 2 modulo 11 is 3.

Public Key Cryptography and RSA

Principles of Public-Key Cryptography

Public-key cryptography, also called **asymmetric cryptography**, is based on the use of two different but mathematically related keys:

- **Public Key – can be shared with everyone.**
- **Private Key – must be kept secret.**

Main Principles

1. **Key Pair**
   - Every user has a public key and a private key.
   - The two keys are mathematically related.
2. **Easy Encryption**
   - It should be computationally easy to encrypt a message using the public key.
3. **Easy Decryption**
   - The intended receiver should be able to decrypt the message efficiently using the private key.
4. **Difficult to Derive Private Key**
   - It should be computationally infeasible to determine the private key from the public key.
5. **Confidentiality**
   - A sender encrypts data using the receiver's public key.
   - Only the receiver's private key can decrypt it.

Sender → Public Key → Encryption → Ciphertext → Private Key → Receiver

6. Authentication and Digital Signature

- A user can use their private key to create a digital signature.
- Others can use the corresponding public key to verify it.

7. Key Distribution

- Public keys can be openly distributed, eliminating the need to secretly share a common key beforehand.

Simple Example

Suppose Alice wants to send a secret message to Bob:

Alice → Bob's Public Key → Encrypt → Ciphertext → Bob's Private Key → Decrypt → Message
Thus, the fundamental idea is:
Public key is shared; private key is kept secret.
These principles form the basis of systems such as RSA, Diffie–Hellman, DSA, and ECC.

Basic Principle

Suppose Alice wants to receive a secret message.

1. Alice generates:

- **a public key** (K_{pub})
- **a private key** (K_{priv})

- Alice publishes her public key.
- Bob encrypts a message using Alice's public key.
- Alice decrypts the ciphertext using her private key.

Alice decrypts the ciphertext using her private key.

Message → Encrypt(Public Key) → Ciphertext
Ciphertext → Decrypt(Private Key) → Message
Anyone can encrypt, but only the private-key holder can decrypt.

Requirements of a Public-Key Cryptosystem

According to the classic requirements:

1. Easy Key Generation
It should be computationally easy to generate a public/private key pair.
2. Easy Encryption
Given the public key and a message, encryption should be efficient.
3. Easy Decryption
The legitimate receiver should be able to decrypt efficiently using the private key.
4. Infeasible to Derive the Private Key
Given the public key, it should be computationally infeasible to determine the private key.
5. Infeasible to Recover Plaintext
Given the public key and ciphertext, recovering the original message without the private key should be infeasible.
6. Reversible Operations
The encryption and decryption operations must be inverses of each other.

Diffie–Hellman Key Exchange

The Diffie–Hellman Key Exchange is a method that allows two parties to establish a shared secret key over an insecure communication channel without previously sharing a secret.
It was introduced by Whitfield Diffie and Martin Hellman in 1976**.**

Basic Idea
Two users, Alice and Bob, want to agree on a secret key. They publicly agree on:

- A large prime number p
- A generator (primitive root) g

These values are not secret.

Steps of Diffie Hellman Key exchange

Step 1: Alice Chooses a Secret
Alice selects a private number a.
She computes

$$A = g^a \bmod p$$

and sends $A$to Bob.

Step 2: Bob Chooses a Secret

Bob selects a private number b.
He computes

$$B = g^b \bmod p$$

and sends $B$to Alice.

Step 3: Alice Computes the Shared Key

Alice computes

$$K = B^a \bmod p.$$

Step 4: Bob Computes the Shared Key

Bob computes

$$K = A^b \bmod p.$$

Since

$$(g^b)^a = (g^a)^b = g^{ab},$$

both obtain the same secret key.

$$K = g^a (\mathrm{mod} p)$$

Numerical
Example Choose:

$$\boldsymbol{p = 23, g = 5}$$

Alice Private key:

$$a = 6$$

Public value:

Alice sends 8.

Bob Private key:

$$b = 15$$

Public value:

$$B = 5^{15} \, mod \, 23 = 19$$

Bob sends 19.

RSA Algorithm

The RSA Algorithm is one of the most widely known public-key cryptosystems. It is used for encryption, digital signatures, and secure key exchange.
The name RSA comes from its inventors:

- Ron Rivest
- Adi Shamir
- Leonard Adleman

RSA has 3 Phases

Phase 01: key generation Phase 02: Encryption
Phase 03: Decryption

1. **Key Generation phase**

Step 1: Choose Two Prime Numbers Select two prime numbers: p **and** q

Step 2: Compute n

$$n = p * q$$

The value n**becomes part of both the public and**

private keys. Step 3: Compute Euler's Totient

$$\phi(n) = (p-1)(q-1)$$

$$\varphi(n) = (p-1)(q-1)$$

Step 4: Choose Public Exponent e
Choose e**such that**

$$1 < e < \phi(n)$$

and

$$\gcd(e, \phi(n)) = 1.$$

Example:

$$e = 7$$

since

$$\gcd(7, 160) = 1.$$

Step 5: Compute Private Exponent d
Find $d$such that

$$d.e \equiv 1 (\mathrm{mod} \phi(n)).$$

Phase 02: Encryption

For a message represented by M, the ciphertext C is calculated as:

$$C = M^e \mathrm{mod}\ n$$

The sender uses the recipient's public key to encrypt the message.

Phase 03: Decryption

The recipient uses the private key to recover the original message:

$$M = C^d \mathrm{mod}\ n$$

Thus:

Plaintext M → Encryption → C → Decryption → M

Elliptic Curve Arithmetic

ECC does not use ordinary multiplication and division for cryptographic keys. Instead, it relies on operations on points.

Given two distinct points:

$$P = (x_1, y_1), Q = (x_2, y_2)$$

their sum $R = P + Q$ is another point on the curve. The slope is:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} (\mathrm{mod} p)$$

Then:

$$x_3 = \lambda^2 - x_1 - x_2 (\mathrm{mod} p)$$

Public-Key Generation

ECC keys are generated as follows:

Private Key
Choose a random integer:

$$d$$

Public Key
Compute:

$$Q = dG$$

where $G$is a publicly known generator point.

The pair is:

- **Private key: d**
- **Public key: Q**

Computing Q from d is easy.
Finding $d$from $Q$is believed to be infeasible.

Elliptic Curve Cryptography (ECC)

Elliptic Curve Cryptography (ECC) is a public-key cryptographic technique that uses the mathematics of elliptic curves over finite fields to provide security. ECC offers strong security with much smaller key sizes than traditional systems such as RSA.

Basic Idea
ECC relies on the difficulty of the **Elliptic Curve Discrete Logarithm Problem (ECDLP)**:
Given:

$$Q = dG$$

where:

- G= publicly known generator point
- d= private key

- Q= public key

it is easy to compute $Q$from d, but computationally infeasible to determine $d$from
Q.

Key Generation

Step 1: Select Domain Parameters
All users agree on:

- An elliptic curve
- A finite field
- A generator point G
- The order n of G

Step 2: Generate Private Key
Choose a random integer:

$$d, \quad 1 \le d \le n - 1$$

Step 3: Generate Public Key
Compute:

$$Q = dG$$

The key pair is:

- Private key: d
- Public key: Q

ECC Operations

Point Addition
Combines two points on the curve: $R = P + Q$

Point Doubling
Adds a point to itself: $2P = P + P$

Scalar Multiplication
Repeated point addition: kP